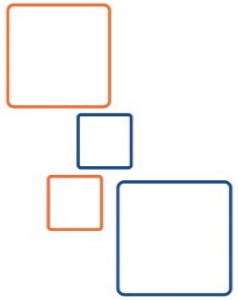
Lecture 7



Presented By
Beshoy Adel

Java™ Education
and Technology Services

Invest In Yourself ,
Develop Your Career

Agenda

- Core Data

Core Data

What Is Core Data?

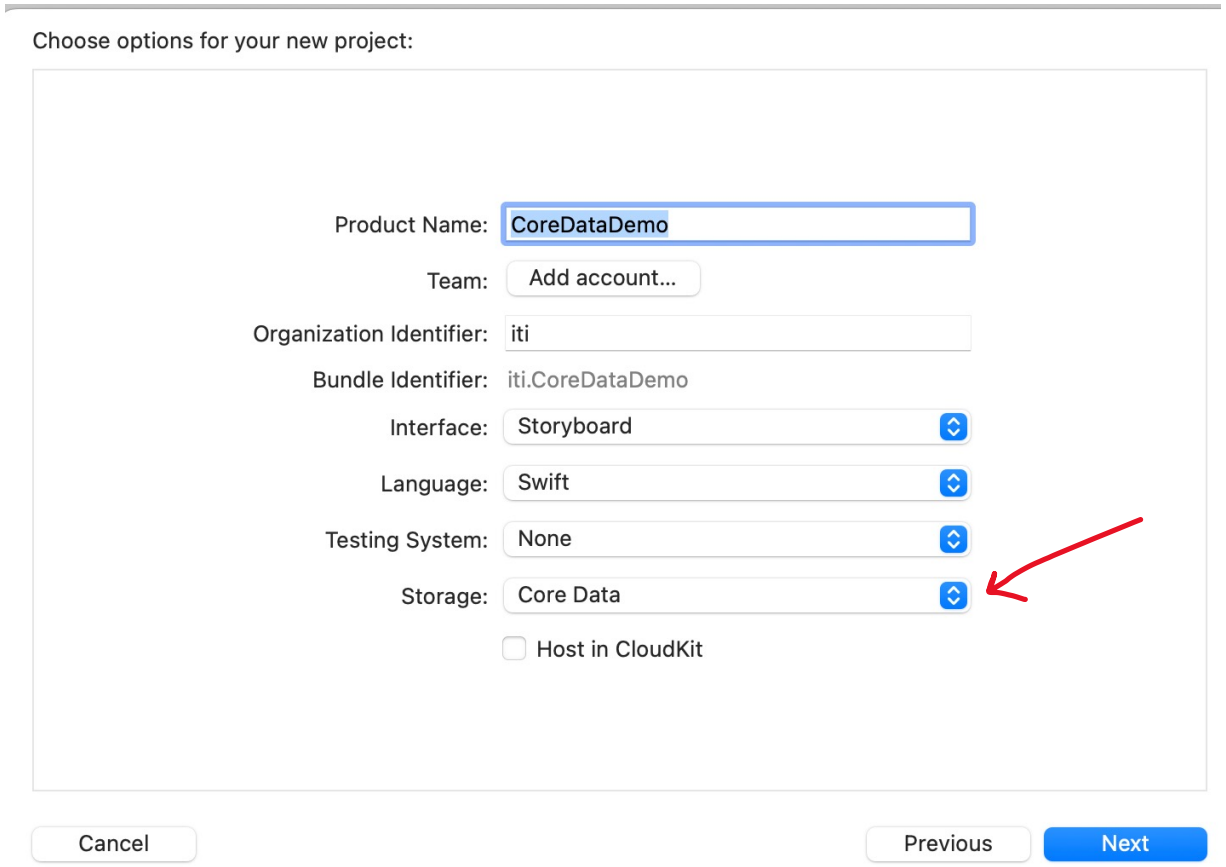
- Core Data is a framework that you use to manage the model layer objects in your application.
- It provides generalized and automated solutions to common tasks associated with object life cycle and object graph management, including persistence.

What Is Core Data? Cont.

- Object-oriented database.
- Very powerful framework in iOS.
- It’s a way of creating an object graph backed by a database.
- Usually backed by SQL.

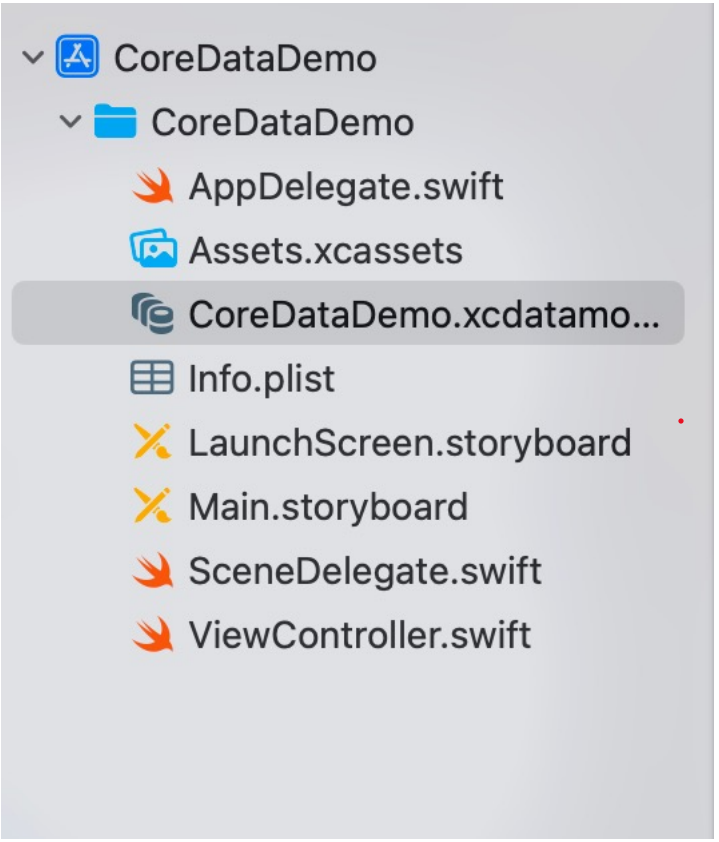
How does it work?

- When you start a new project in Xcode and open the template selection dialog, select the Use Core Data checkbox.



How does it work? Cont.

- A source file for the Core Data model is created as part of the template.
- That source file will have the extension .xcdatamodeld.

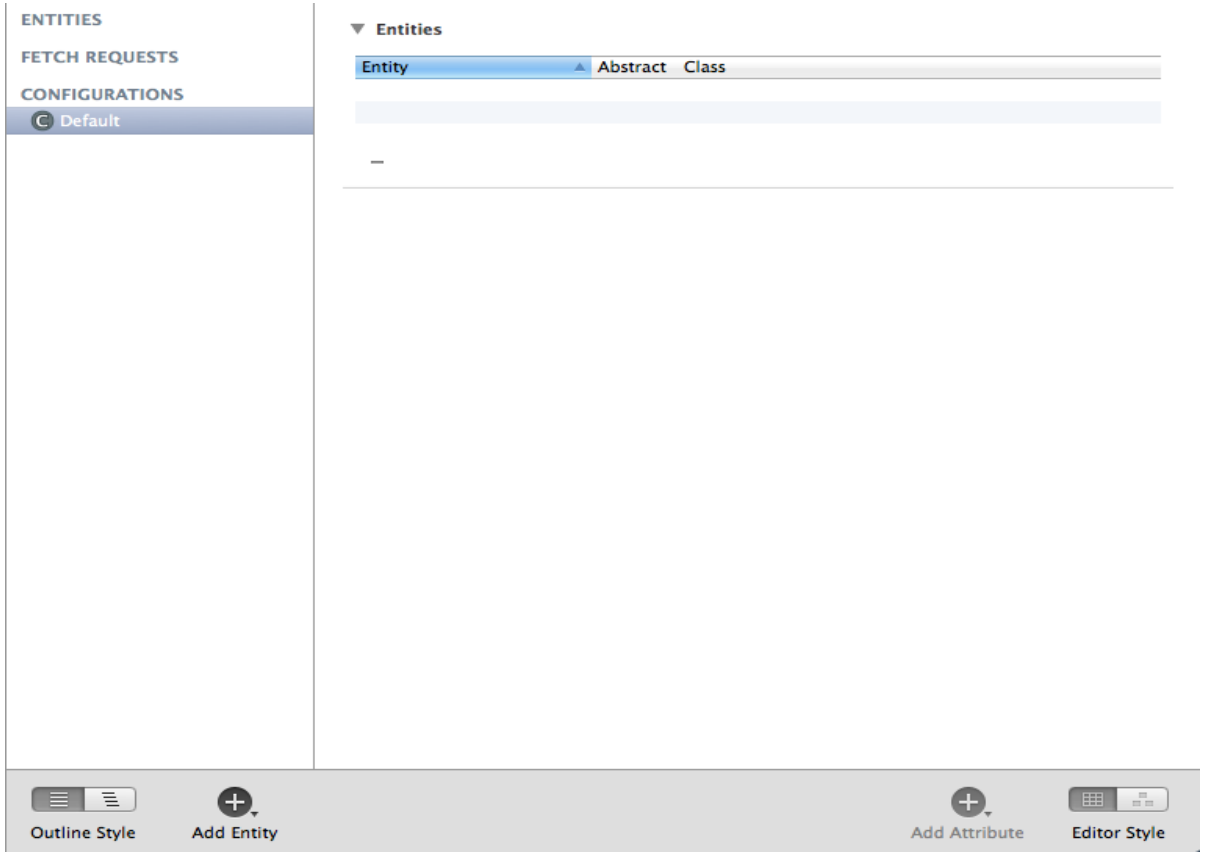


Entity

- An ***entity*** is a class definition in Core Data.
- In our example the entity is a ***Movie***.
- In a relational database, an entity corresponds to a table.

Create an entity

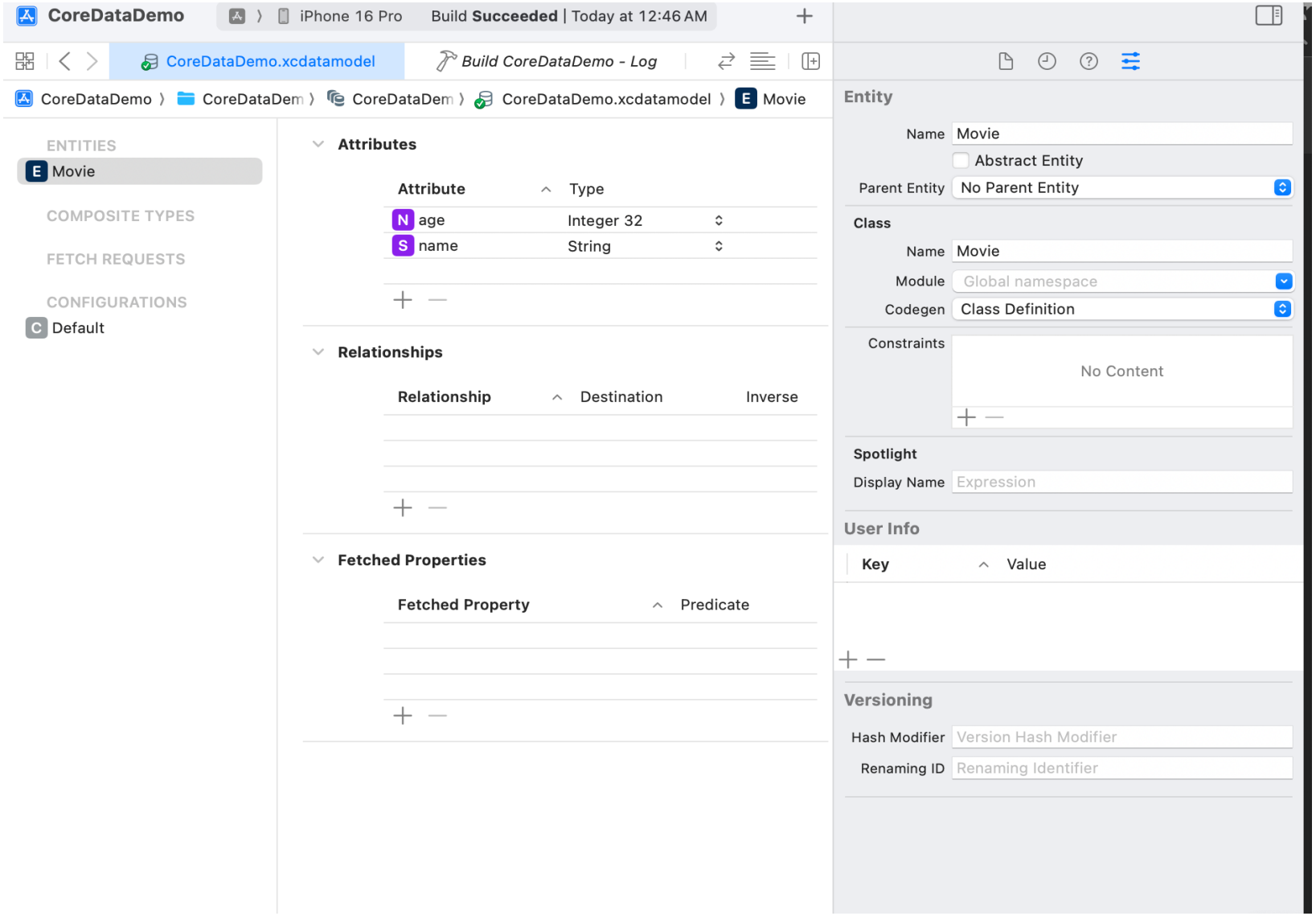
- Select the file in the navigator area to display the Core Data model editor



Create an entity Cont.

- Click Add Entity.
- A new untitled entity will appear in the Entities list in the navigator area.
- Select the new untitled entity.
- In the Entity pane of the Data Model inspector, enter the name of the entity, and press Return.

Create an entity Cont.



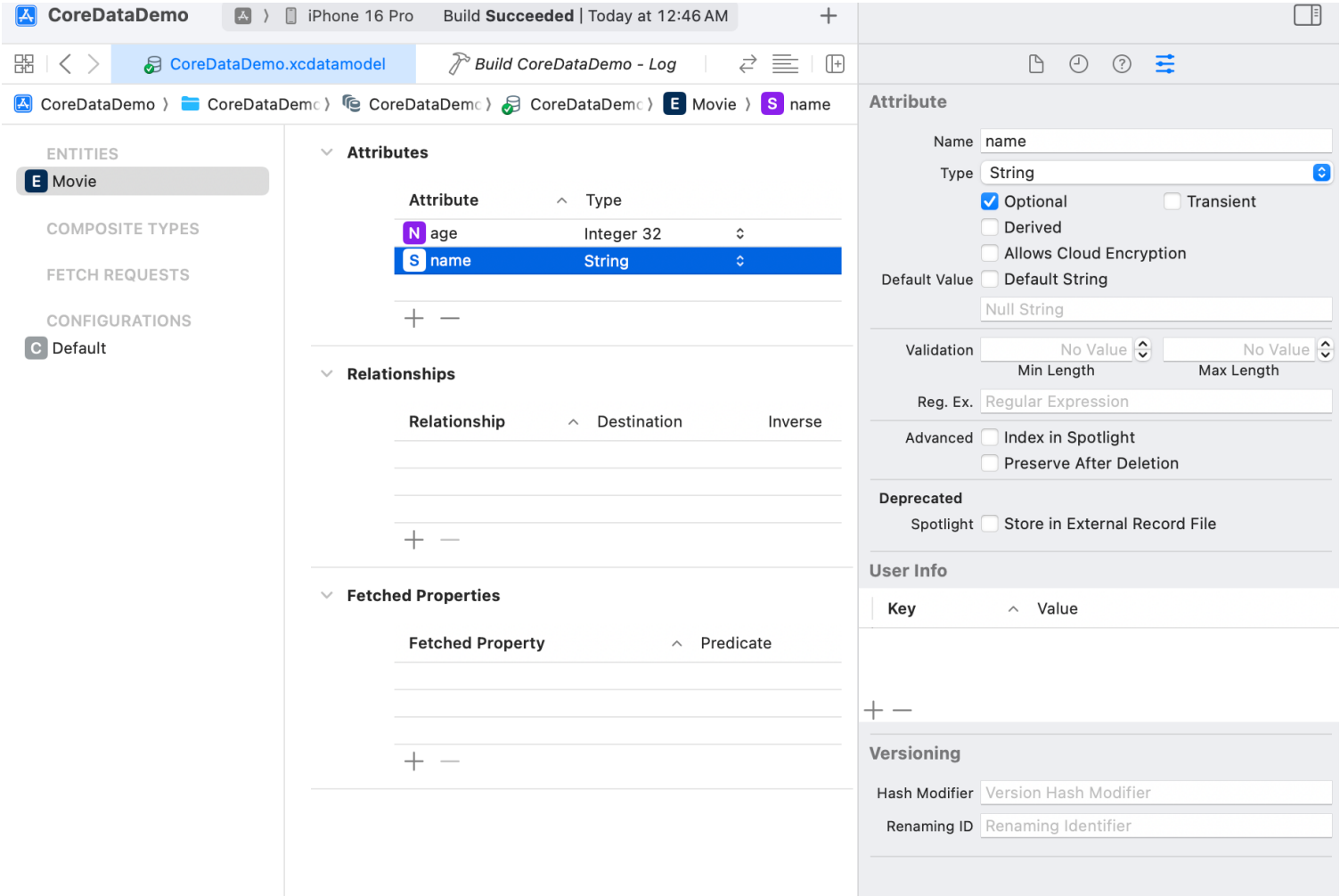
Attribute

- An ***attribute*** is a piece of information attached to a particular entity.
- For example, a ***Movie*** entity could have attributes for the movie's title, image, rating and releasedYear.
- In a database, an attribute corresponds to a particular field in a table.

Create attributes for the entity

- With the new entity selected, click the plus(+) sign at the bottom of the appropriate section.
- A new untitled attribute is added under the Attributes section of the editor area.
- Select the new untitled property.
- The property settings are displayed in the attributes pane of the Data Model inspector.
- Give the property a name and press Return.
- The attribute information appears in the editor area.

Create attributes for the entity Cont.



Saving to Core Data

- Import the Core Data module at the top of ***ViewController.swift***
- You may have had to link frameworks manually in your project's Build Phases if you've worked with Objective-C frameworks.
- In Swift, a simple import statement is all you need to start using Core Data APIs in your code.

```
import UIKit
import CoreData
```

Saving to Core Data Cont.

```
//1
let appDelegate = UIApplication.shared.delegate as!
    AppDelegate

//2
let managedContext = appDelegate.persistentContainer.
    viewContext

//3
let entity = NSEntityDescription.entity(forEntityName:
    "Movie", in: managedContext)

//4
let movie = NSManagedObject(entity: entity!,
    insertInto: managedContext)

movie.setValue("Movie", forKey: "title")
movie.setValue(2.3, forKey: "rating")
movie.setValue(2007, forKey: "releaseYear")

do{
    //5
    try managedContext.save()
}catch let error as NSError{

    print(error)

}
```

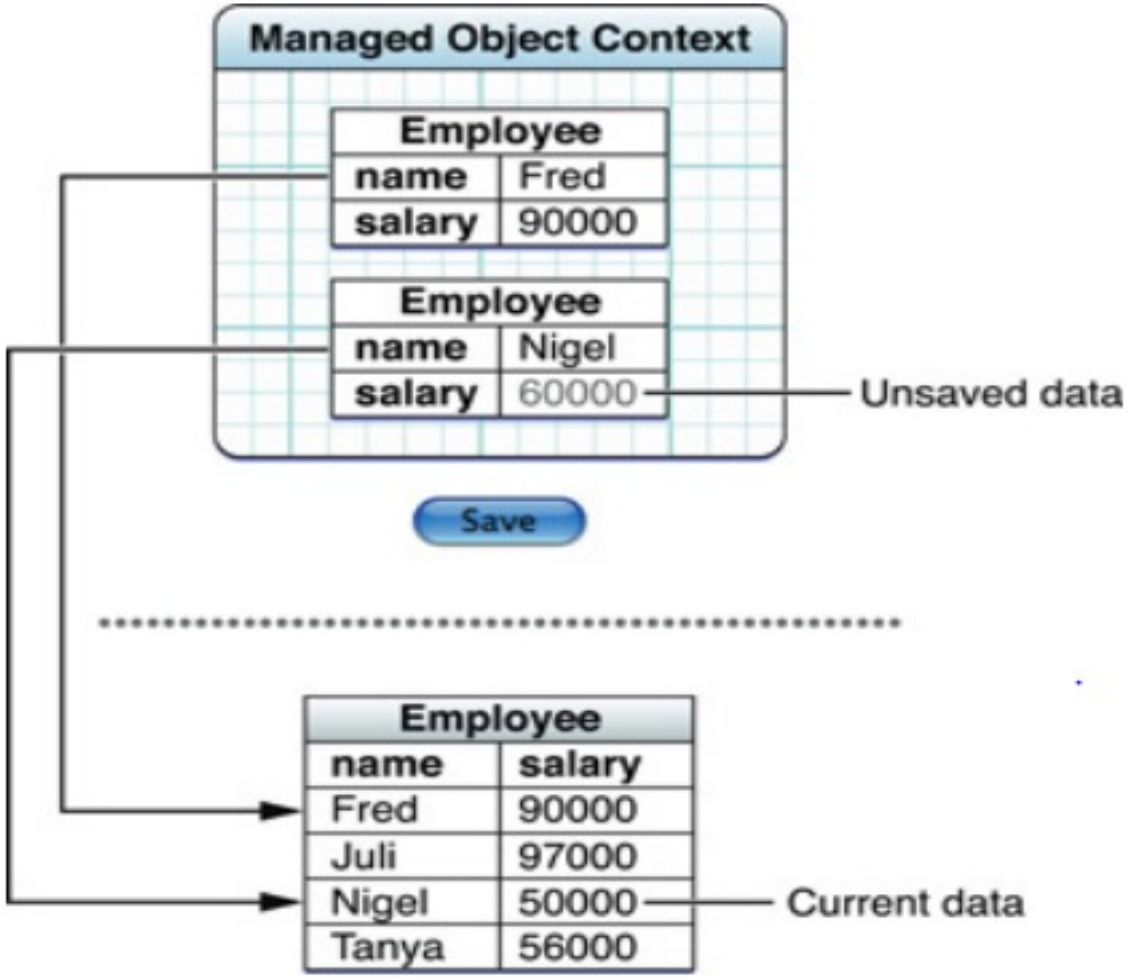
Managed Object Context

- A managed object context is an instance of NSManagedObjectContext.
- Its primary responsibility is to manage a collection of managed objects.
- These managed objects represent an internally consistent view of one or more persistent stores.

Managed Object Context Cont.

- The context is the central object in the Core Data stack.
- It’s the object you use to create and fetch managed objects.
- A context is connected to a parent object store.
- This is usually a persistent store coordinator.
- the context asks its parent object store to return those objects that match the fetch request.
- Changes that you make to managed objects are not committed to the parent store until you save the context.

Managed Object Context Cont.



Fetching from Core Data

```
//1
let appDelegate = UIApplication.shared.delegate as!
    AppDelegate

//2
let managedContext = appDelegate.persistentContainer.
    viewContext


//3
let fetchRequest = NSFetchRequest<NSManagedObject>
    (entityName: "Movie")

//4
let myPredicate = NSPredicate(format: "title == %@",
    "Movie 1")
fetchRequest.predicate = myPredicate

do{

     //5
     movies = try managedContext.fetch(fetchRequest)
}catch let error as NSError{

    print(error)
}
```

Deletion from Core Data

```
//1
let appDelegate = UIApplication.shared.delegate as!
    AppDelegate

//2
let managedContext = appDelegate.persistentContainer.
    viewContext

//3
managedContext.delete((movies?[indexPath.row])!)

movies?.remove(at: indexPath.row)


do{
   //4
    try managedContext.save()
}catch let error as NSError{

    print(error)

}
```

Core Data Demo

Assignments

1.product APP (new)

- Update your product app when you add **+** button on the navigation bar which present view allow you to add new product and when you press **Done** button, The product will store in core data and the list view updated with the new product.
- When run this application again the list display the stored products.

2.product APP (cashing)

- Update your product app to Save all retrieved data using core data.
- If the application is online the data retrieved from the webservice and update the stored data.
- If the application is offline the data retrieved from the core data.
- Display the product detail in another view controller when select it in the collection view.